EXPERIMENT NO.7

Name: Vaishnal Mali
Class:D20A
Roll No:32
Batch:B

Aim:Implement the embedding wallet (Metamask) and transaction using Solidity

Theory:

1. What is MetaMask?

MetaMask is a software cryptocurrency wallet used to interact with the Ethereum blockchain and other EVM-compatible networks (like Polygon, BSC, and Avalanche).

It is primarily known as a **"bridge"** because it allows users to access Decentralized Applications (dApps) directly through a regular web browser (like Chrome or Firefox) without having to run a full Ethereum node.

Core Theoretical Functions:

- **Identity Management:** It generates and manages your private keys locally on your device (not on a central server), giving you full ownership of your digital identity.
- **Web3 Injection:** It injects a JavaScript library called **web3.js** or **ethers.js** into the websites you visit. This allows the website to "talk" to the blockchain.
- **Transaction Signing:** When you want to perform an action on a dApp (like buying an NFT), MetaMask "pops up" to let you review and digitally sign the transaction before it is sent to the network.
- **Network Switching:** It allows users to switch between the "Mainnet" (real money) and various "Testnets" (fake money).

2. What is a Testnet?

A **Testnet** (Test Network) is an alternative blockchain that acts as an exact replica of the main blockchain (Mainnet) but is used specifically for **experimenting, testing, and debugging.**

In blockchain theory, deploying code directly to the Mainnet is risky and expensive because every transaction costs real money (Gas fees) and bugs can lead to permanent financial loss. Testnets solve this by providing a "sandbox" environment.

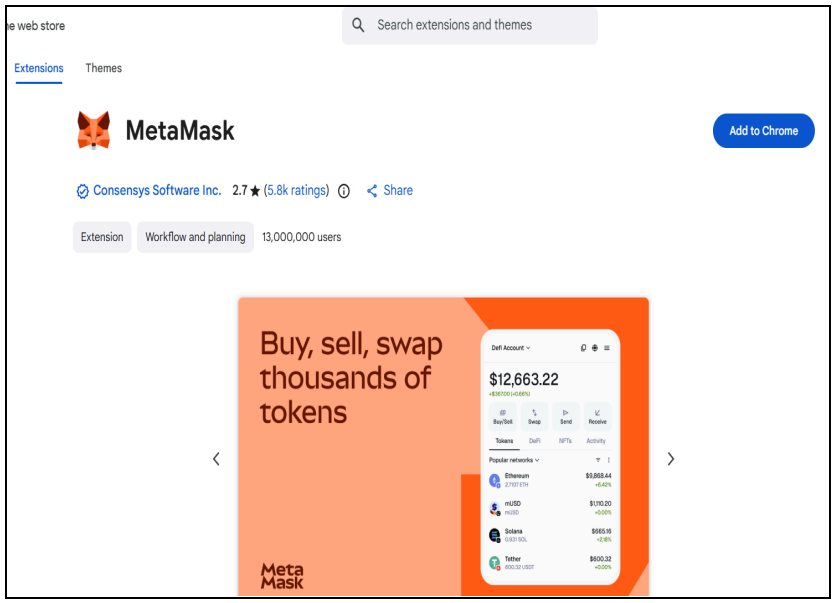
Key Characteristics:

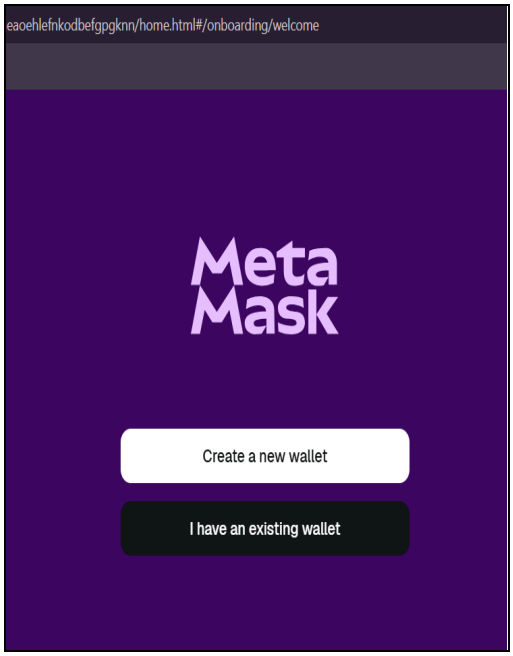
- **No Real Value:** The tokens used on a testnet (e.g., Sepolia ETH) have **zero real-world market value.** You can get them for free from "Faucets."
- **Identical Environment:** It uses the same rules, consensus mechanisms (like Proof of Stake), and smart contract logic as the Mainnet.
- **Safe Playground:** If a developer's smart contract has a bug that crashes the system or "leaks" funds, no real money is lost.
- **Public Access:** Most testnets are public, meaning anyone can join and test their applications before the official launch.

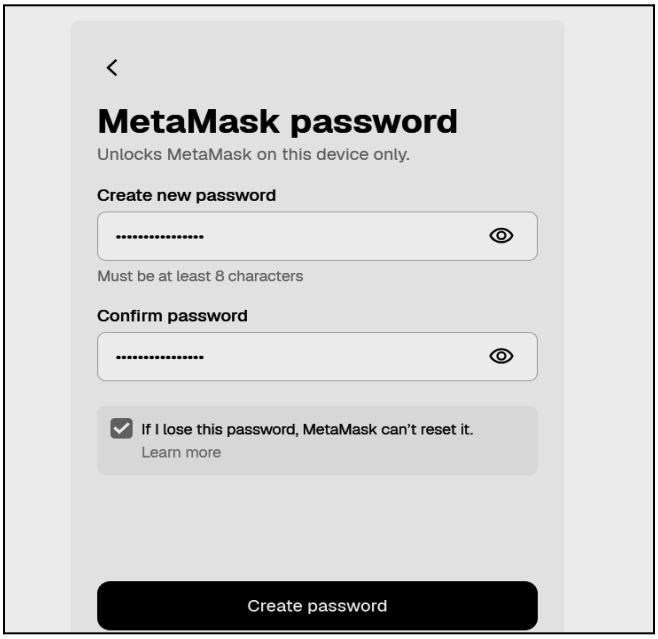
Implementation:

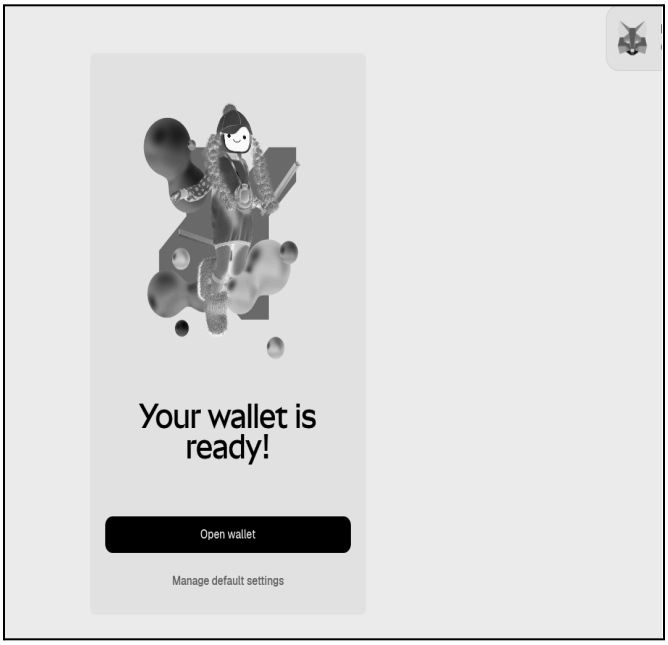
Step 1: Install Metamask

Follow this link to install the Metamask Google Chrome Extension, and also create the wallet







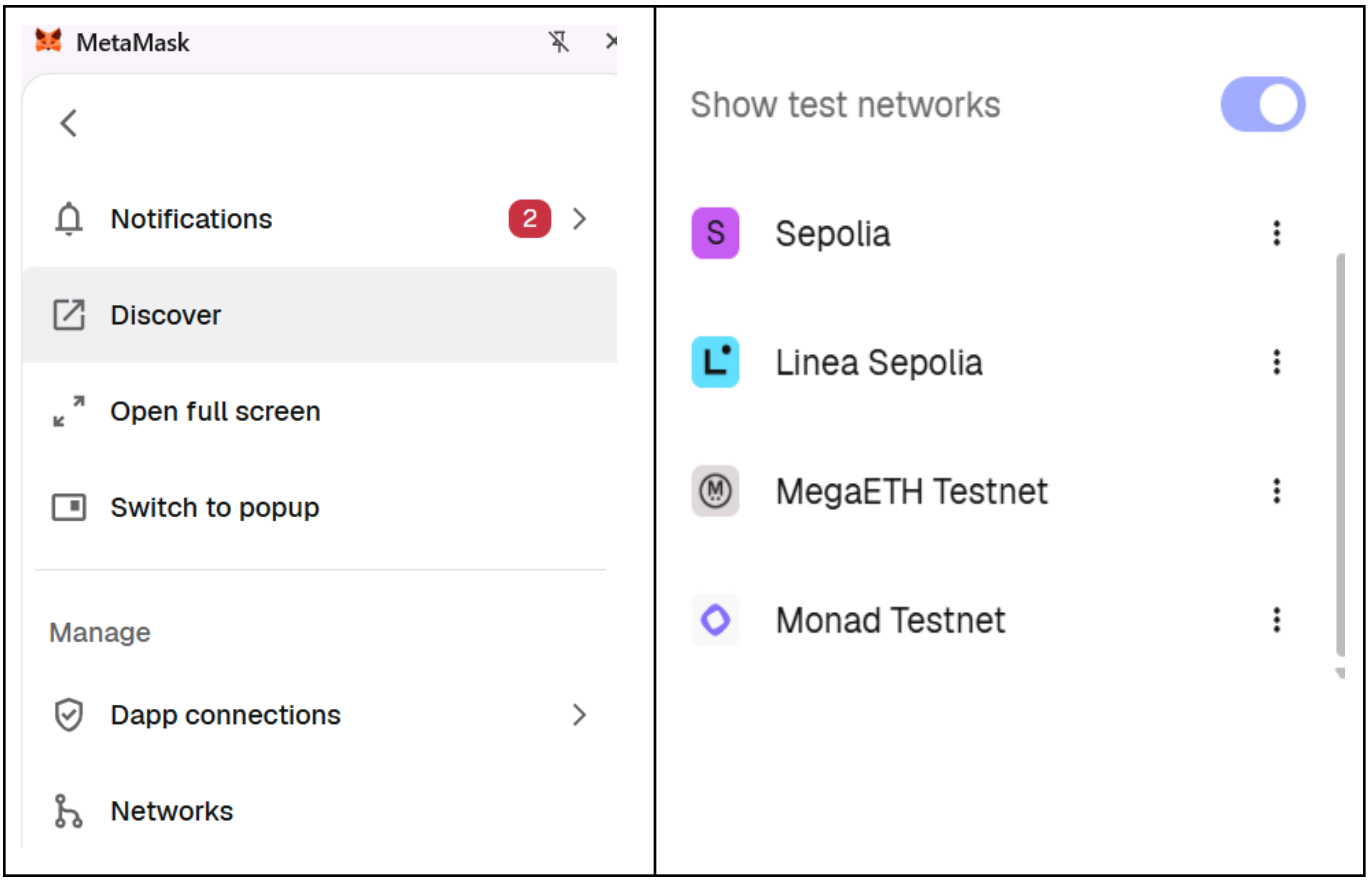


Note :

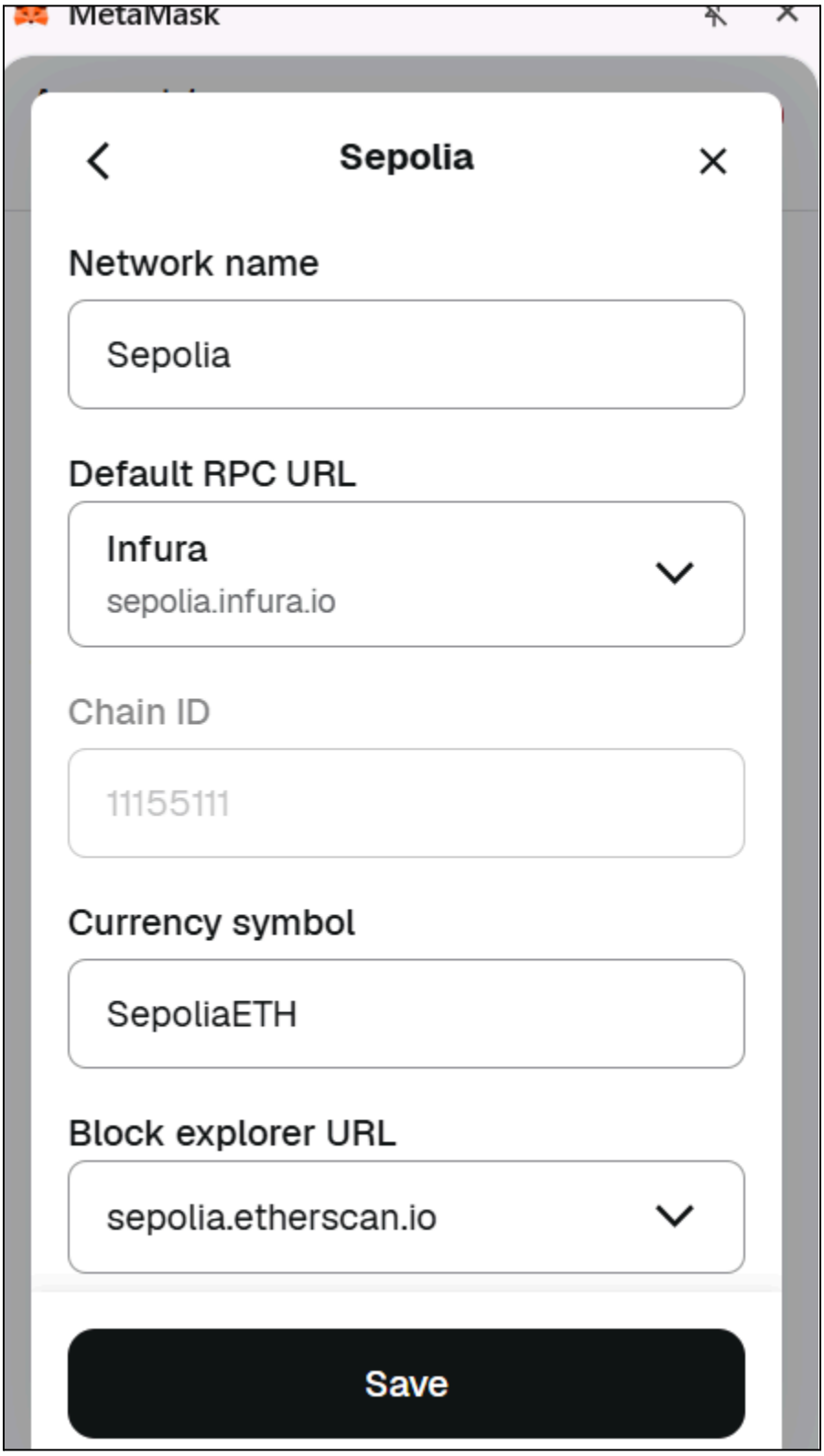
- While creation of the Metamask Wallet, you will be asked to select 12 pass phrase
- Keep the passphrase safley
- It becomes handy to recover your password

Step 2: Enable Test Networks

1. By default, MetaMask shows the "Ethereum Mainnet" (real money).
2. Click the network dropdown (top left) and toggle **Show test networks**.
3. Select **Sepolia**.



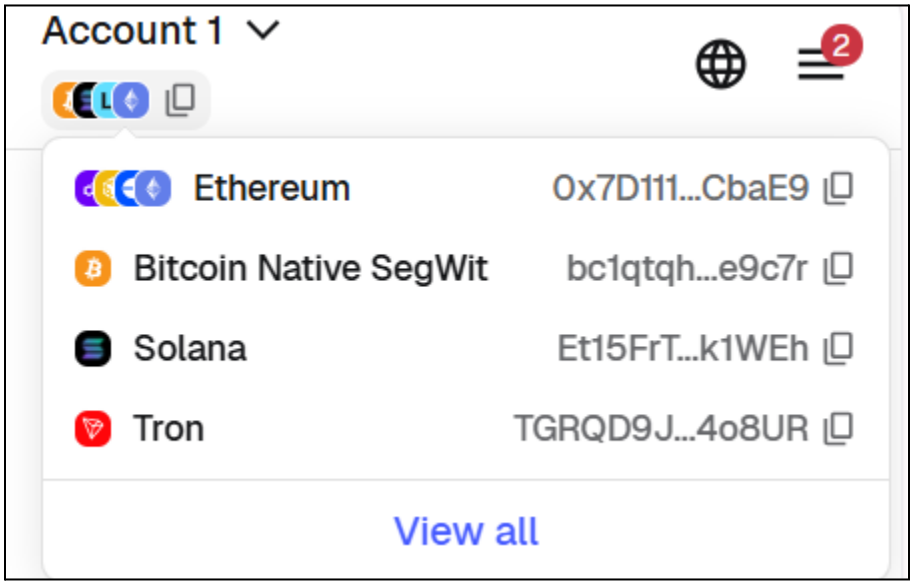
Step 3: Configuring the Sepolia Test network



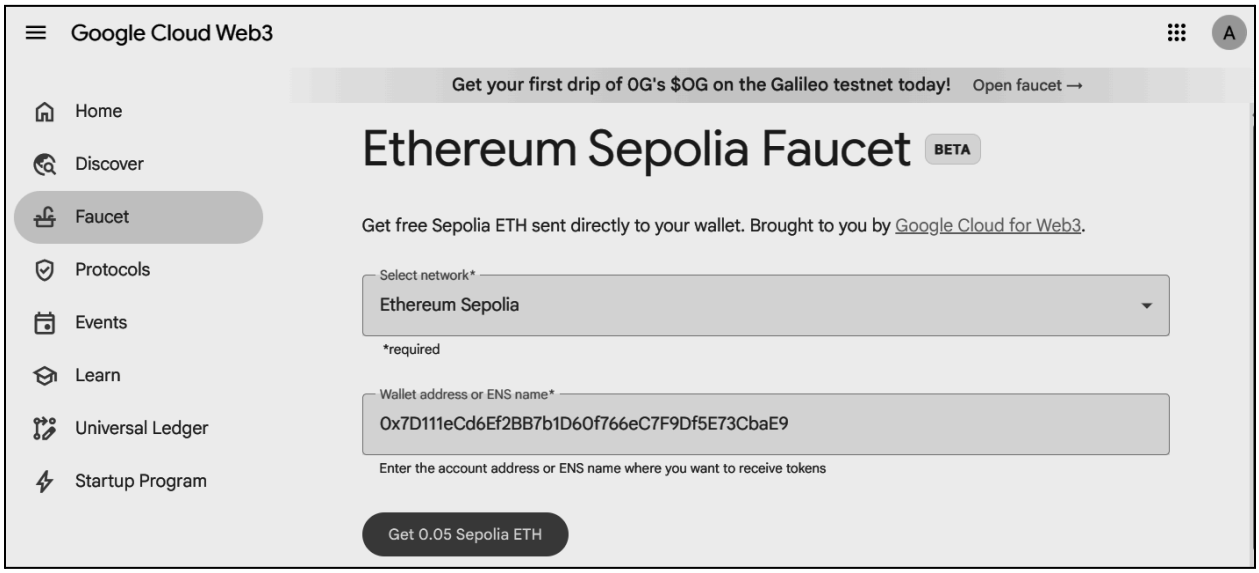
Step 4: Funding the Wallet

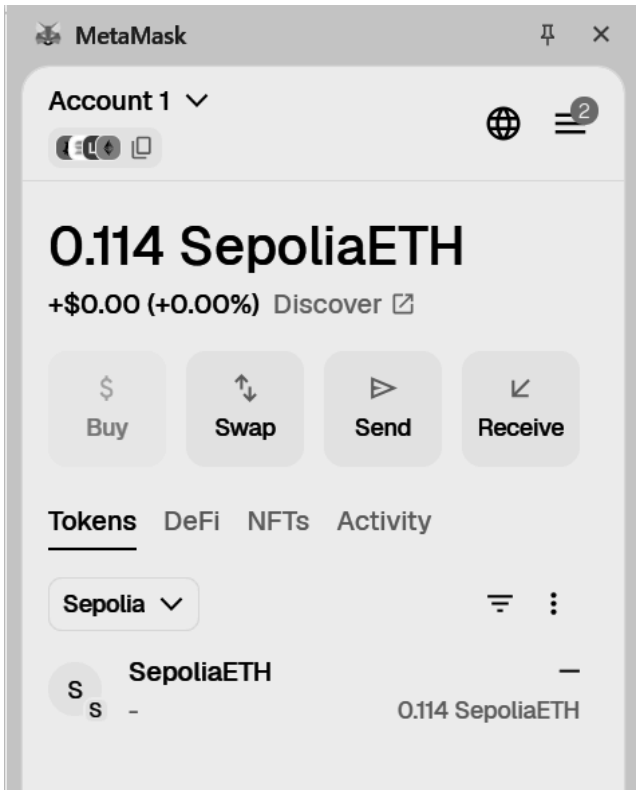
Every action on the blockchain requires a fee called **Gas**. On the Sepolia network, we use "Test ETH" which is free.

1. **Copy Your Address**: Click your account name at the top of MetaMask (it starts with `0x...`) to copy your public address.

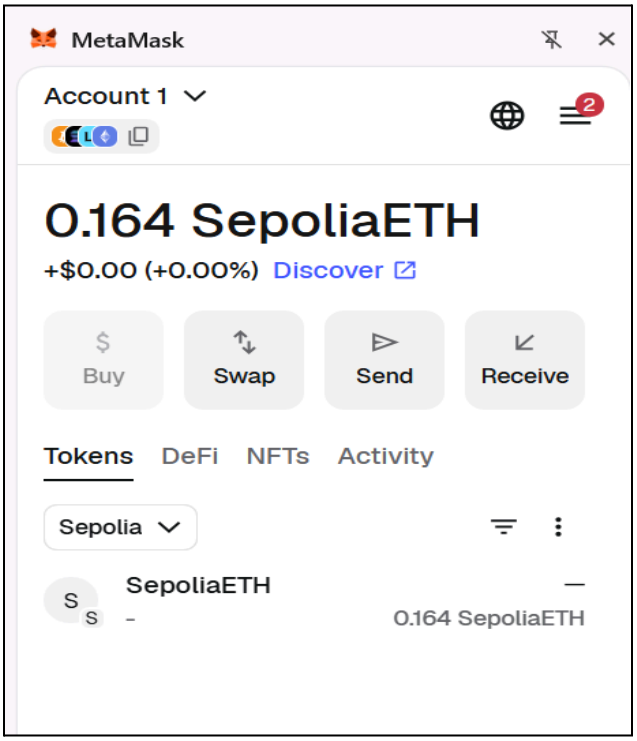


2. **Visit a Faucet**: Use the [Google Cloud Sepolia Faucet]() to procure SepoliaETH by pasting the wallet address as shown



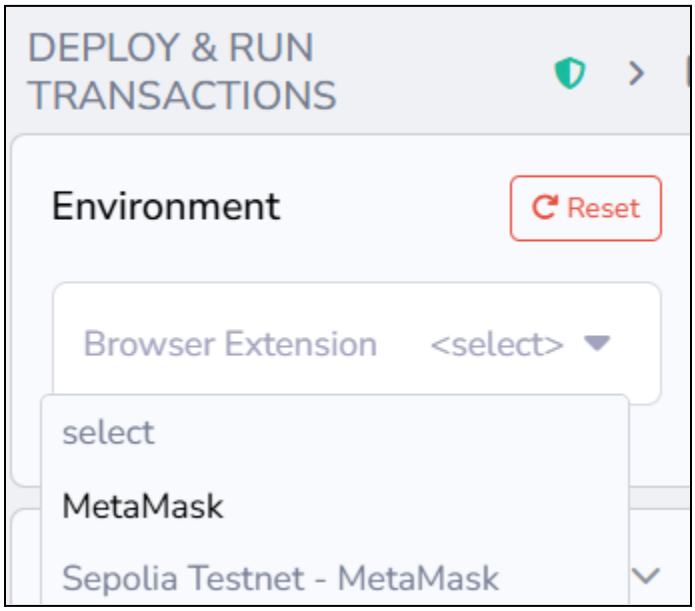


3.**Verify**: Your MetaMask balance should update within 30 seconds.

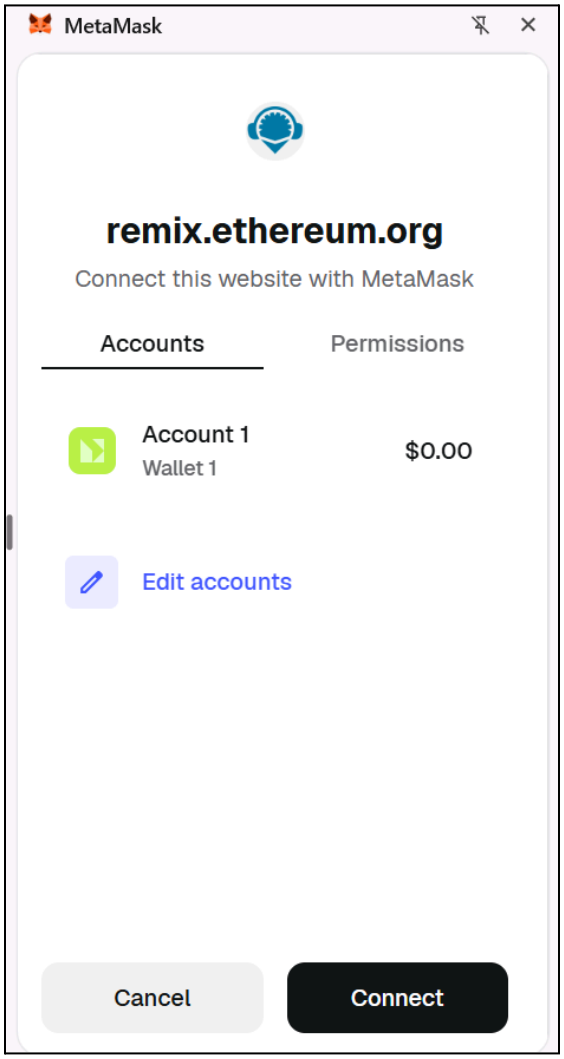


Step 5: Connect MetaMask to Remix IDE

1. Open Remix IDE.
2. Go to the **Deploy & Run Transactions** tab.
3. In the **Environment** dropdown, select **Injected Provider - MetaMask**.

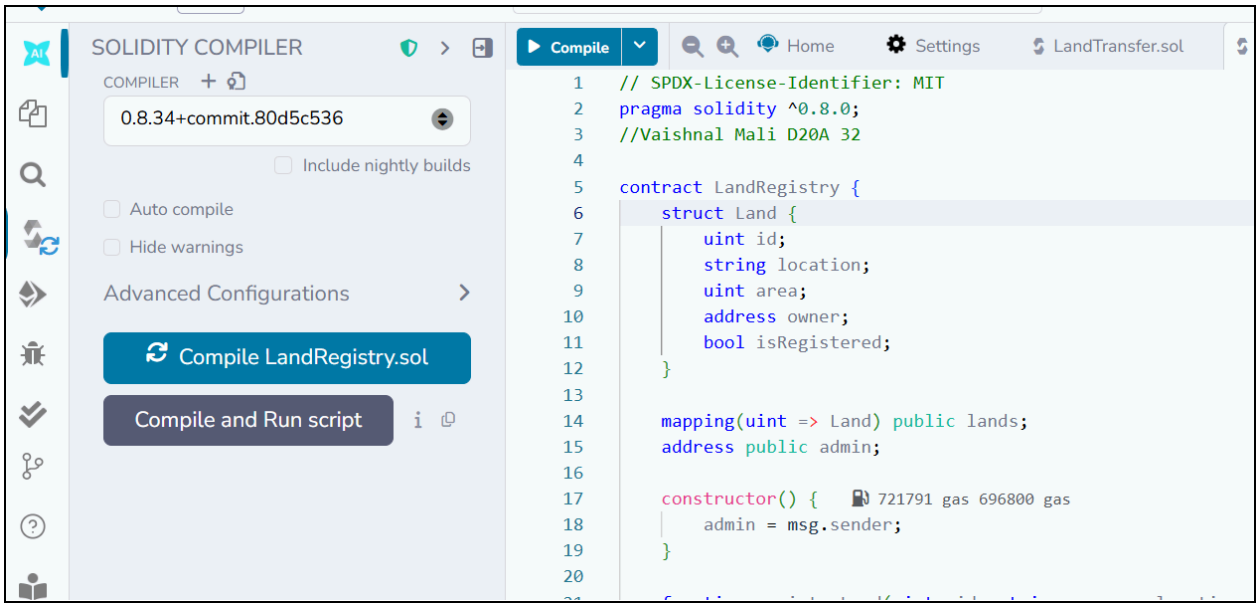


A MetaMask popup will appear; click **Connect** to link your `0x7D11...` account to Remix.



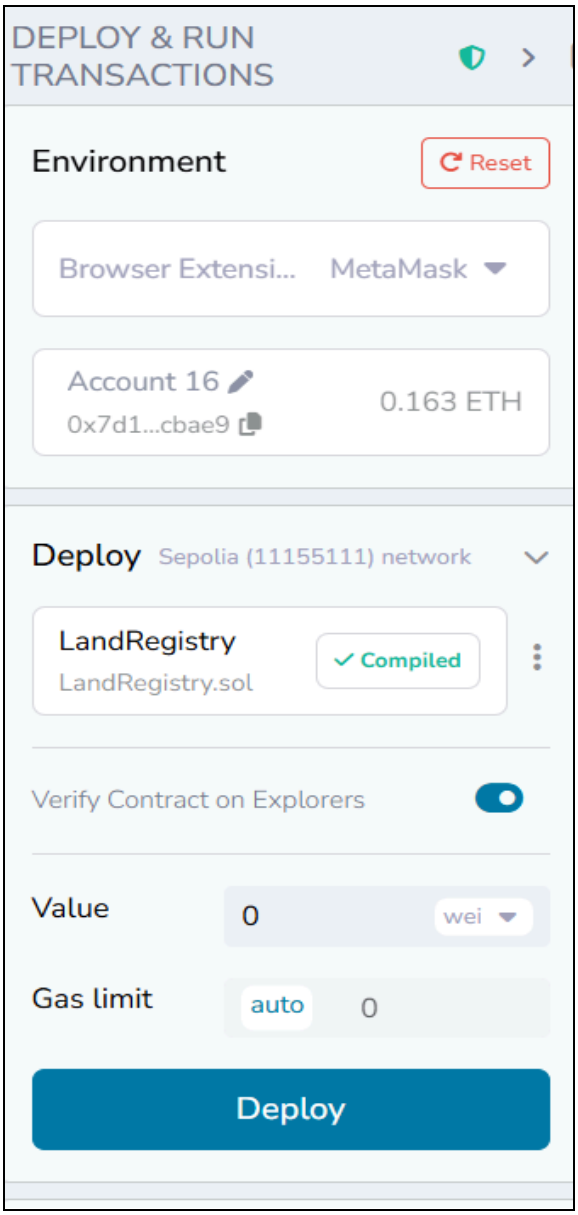
Step 6: Create and Compile the Contract

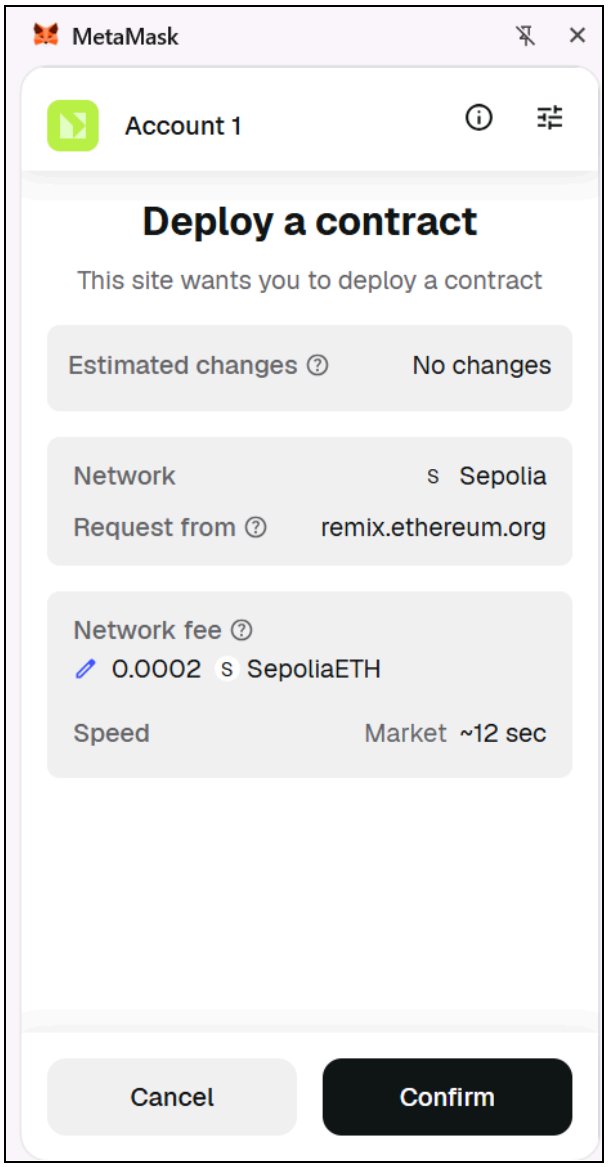
1. Open your LandRegistry.sol file.
2. Go to the **Solidity Compiler** tab.
3. Select Compiler 0.8.0 (or the version in your code).
4. Click **Compile LandRegistry.sol**.



Step 7: Deploy and Verify the transaction

1. Go back to the **Deploy & Run Transactions** tab.
2. Ensure **Injected Provider** is still selected (you should see "Custom (11155111) network").
3. Click the orange **Deploy** button.
4. **MetaMask Action:** A popup will show the gas fee. Click **Confirm**.
5. **Wait:** Look at the bottom terminal. Wait for the green checkmark that says "transacted."





After clicking on deploy button

Sepolia Testnet

Search by Address / Txn Hash / Block / Token

Transaction Details

Overview | State

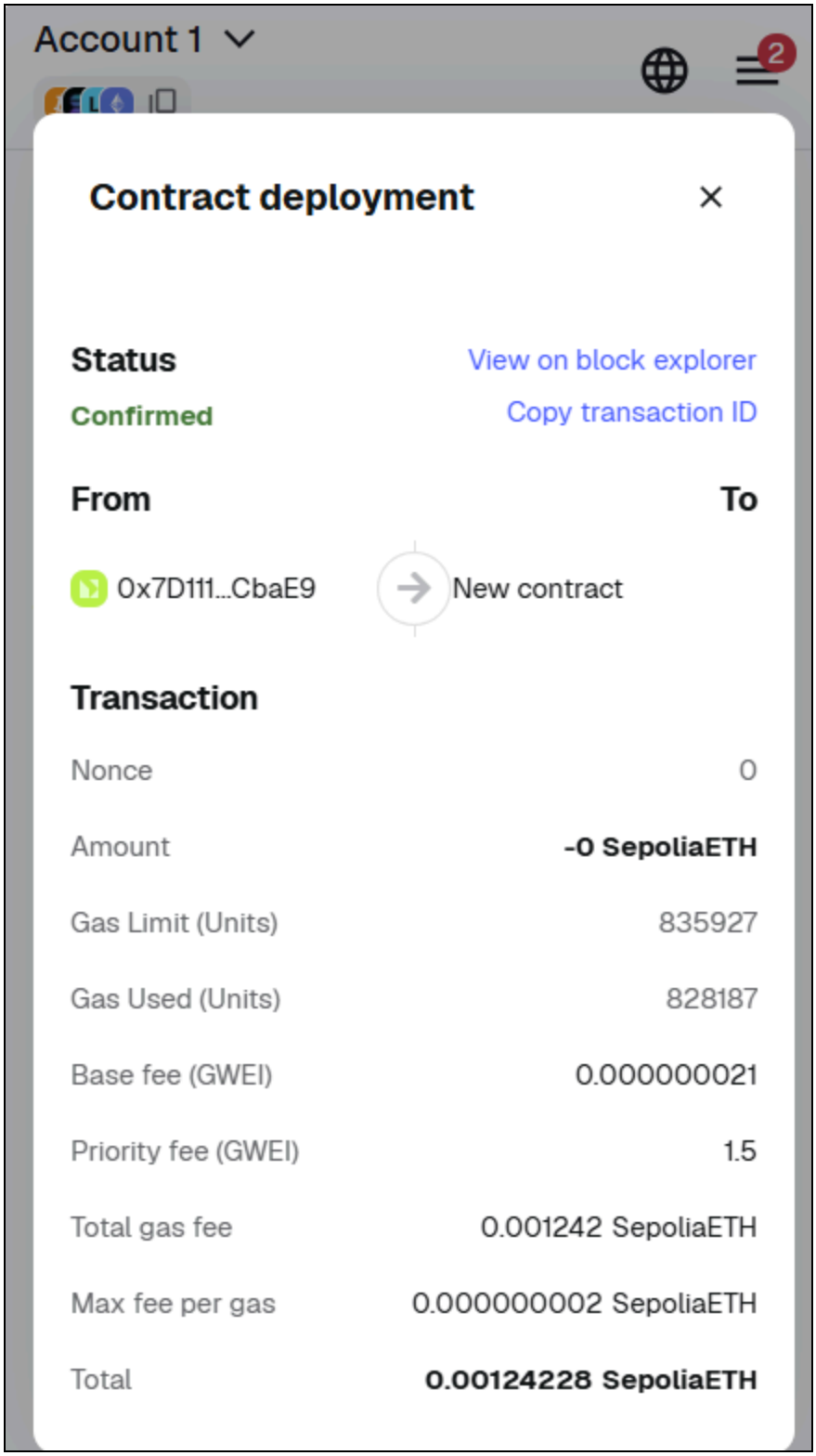
TRANSACTION ACTION

Call 0x60806040 Method by 0x7D111eCd...5E73CbaE9

[This is a Sepolia **Testnet** transaction only]

Transaction Hash:	0x5ddffc83a10bff63281513255d95206694b867c481de576cc0408c4666d12f4f
Status:	Success
Block:	10527926 1 Block Confirmation
Timestamp:	10 secs ago (Mar-26-2026 08:18:00 PM +UTC) UTC
From:	0x7D111eCd6Ef2BB7b1D60f766eC7F9Df5E73CbaE9
To:	[0xa96899f7bde961cd19c979285124bb70725ee133 Created]
Value:	0 ETH
Transaction Fee:	0.001242280517391927 ETH
Gas Price:	1.500000021 Gwei (0.000000001500000021 ETH)

Transaction is verified on Sepolia Etherscan


Account 1
2
Contract deployment
Status
View on block explorer
Confirmed
Copy transaction ID
From
To
0x7D111...CbaE9
New contract
Transaction
Nonce
0
Amount
-0 SepoliaETH
Gas Limit (Units)
835927
Gas Used (Units)
828187
Base fee (GWEI)
0.000000021
Priority fee (GWEI)
1.5
Total gas fee
0.001242 SepoliaETH
Max fee per gas
0.000000002 SepoliaETH
Total
0.00124228 SepoliaETH

```
[block:10527933 txIndex:35]  from: 0x713...99CC2
to: 0xe96F6350d2059BA2E357fbDB3fa173eC18c6af16 0xe96...6af16 value: 0 wei
data: 0x786...b844b logs: 2 hash: 0x190...619a1                                   Debug

status                    1 Transaction mined and execution completed

transaction hash          0x0e1dd86e32bf3c041aa2af07be77d541538950207faada44c3cf0643c90a4e89

from                      0x7138a608F5CEE0db6FA98f1212c1D37706399CC2

to                        0xe96F6350d2059BA2E357fbDB3fa173eC18c6af16

decoded output             -

logs                      []

raw logs                  [
                            {
                              "_type": "log",
                              "address": "0x433EBAFc4A25A5377b0EA8602088AB896509dd37",
                              "blockHash": "0x19073956bd1576d91974899cf334655e1c712bcaee919e6c1328b5b6d2f619a1",
                              "blockNumber": 10527933,
                              "data": "0x",
                              "index": 168,
```

```
view on Etherscan    view on Blockscout
[Verification] Contract deployed. Checking explorers for registration...
Etherscan verification skipped: API key not provided.
Please input the API key in Remix Settings - Connected Services OR Contract Verification Plugin Settings.
[Blockscout] Verification submitted. Awaiting confirmation...
[Sourcify] Verification submitted. Awaiting confirmation...
[Routescan] Verification submitted. Awaiting confirmation...
[Sourcify] Verification Successful!  View Code
[Blockscout] Verification Successful!  View Code
```

Step 8: Interact with the Contract

1. In Remix, under "Deployed Contracts," expand LandRegistry.
2. Run registerLand with ID 1, Location "Mumbai", and your address.
3. **MetaMask Action:** You must click **Confirm** in MetaMask again because this action changes the blockchain state.

non-payable **registerLand** uint256, str...

1

"Mumbai:

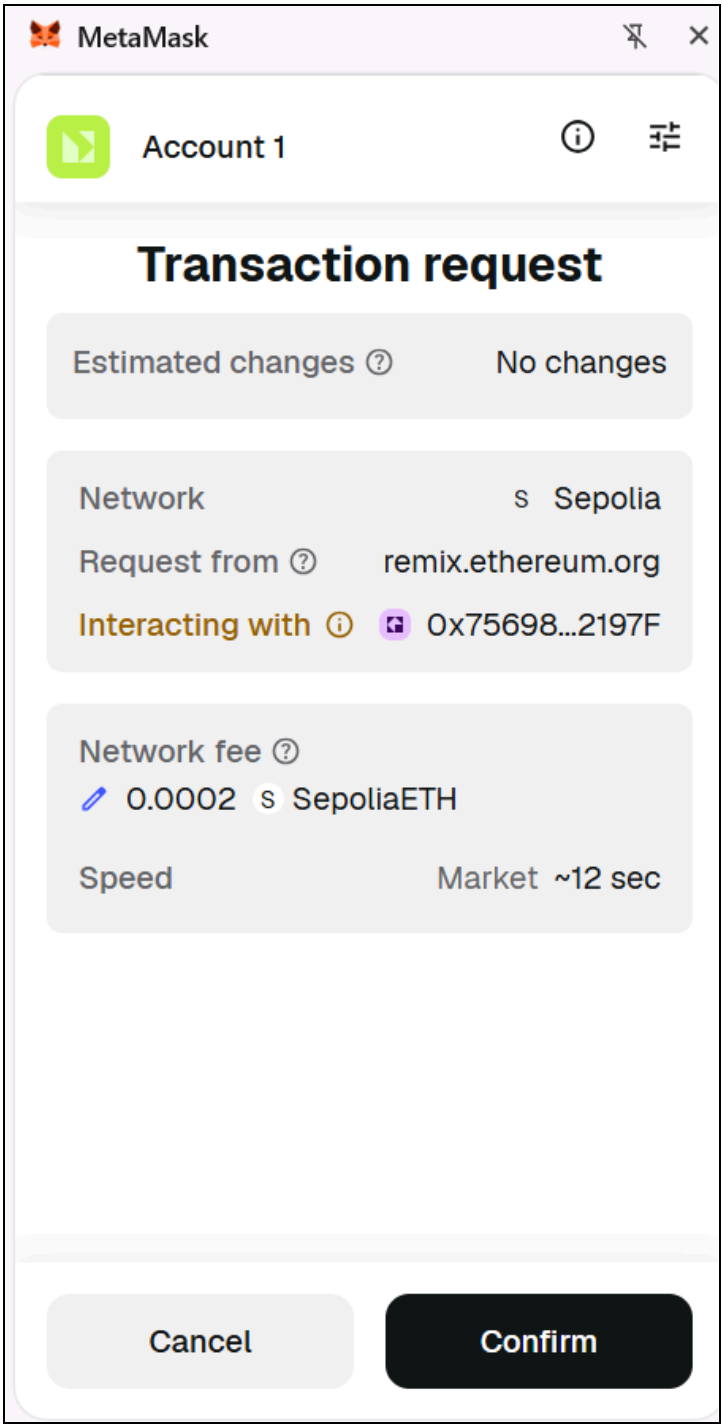
500

0x7D111eCd6Ef2BB7b1D60f766eC

Value 0 wei

Gas limit auto 0

Transact


MetaMask
Account 1
Transaction request
Estimated changes
No changes
Network
Sepolia
Request from
remix.ethereum.org
Interacting with
0x75698...2197F
Network fee
0.0002
SepoliaETH
Speed
Market
~12 sec
Cancel
Confirm

Contract interaction

Status

Confirmed

View on block explorer

Copy transaction ID

From

0x7D111...CbaE9

To

0x75698...2197F

Transaction

Nonce	2
Amount	**-0 SepoliaETH**
Gas Limit (Units)	118966
Gas Used (Units)	115835
Base fee (GWEI)	0.000000028
Priority fee (GWEI)	1.5
Total gas fee	0.000174 SepoliaETH
Max fee per gas	0.000000002 SepoliaETH
Total	**0.00017375 SepoliaETH**

Conclusion:-

This experiment helped in understanding how smart contracts automate processes and execute securely on the Ethereum blockchain. Using Solidity and Remix IDE, the development, deployment, and interaction with smart contracts become simple and efficient, forming the foundation for decentralized applications.